17. Command Design Pattern in Java

17.1 Introduction

The **Command Design Pattern** is a **behavioral pattern** that turns a request into a stand-alone object, allowing you to parameterize methods with different requests, queue them, and support undoable operations.

17.2 Problem it Solves

You want to decouple the object that invokes the operation from the one that knows how to perform it. This is especially useful when commands need to be queued, logged, or undone.

17.3 What is Command Pattern?

It encapsulates a request as an object, thereby letting you parameterize clients with queues, requests, and operations.

17.4 Real-World Analogy

A restaurant waiter takes an order (command) from a customer and submits it to the kitchen. The waiter doesn't need to know how to cook it.

17.5 Structure

```
package com.mannemlokesh;

public interface Command {
	void execute();
}

class Receiver {
	public void action() {
		System.out.println("Action performed!");
	}
}

class ConcreteCommand implements Command {
	private Receiver receiver;

	public ConcreteCommand(Receiver receiver) {
		this.receiver = receiver;
	}

	public void execute() {
		receiver.action();
	}
}

class Invoker {
	private Command command;

	public void setCommand(Command command) {
		this.command = command;
	}

	public void invoke() {
```

```
            command.execute();
        }
}
```

17.6 Benefits

- Decouples sender from receiver
- Easy to add/modify commands
- Supports undo/redo functionality

17.7 Implementation Steps

1. Create a Command interface
2. Create concrete command classes
3. Store the receiver in the command
4. Use invoker to execute commands

17.8 Examples

Example1: Light Switch

```
package com.mannemlokesh.example1;

interface Command {
        void execute();
}

class Light {
        public void on() {
                System.out.println("Light is ON");
        }

        public void off() {
                System.out.println("Light is OFF");
        }
}

class LightOnCommand implements Command {
        private Light light;

        public LightOnCommand(Light light) {
                this.light = light;
        }

        public void execute() {
                light.on();
        }
}

class LightOffCommand implements Command {
        private Light light;

        public LightOffCommand(Light light) {
                this.light = light;
        }

        public void execute() {
                light.off();
```

```
	}
}

class Switch {
	private Command onCommand;
	private Command offCommand;

	public Switch(Command on, Command off) {
		this.onCommand = on;
		this.offCommand = off;
	}

	public void turnOn() {
		onCommand.execute();
	}

	public void turnOff() {
		offCommand.execute();
	}
}

public class Example1LightSwitchClient {
	public static void main(String[] args) {
		Light light = new Light();
		Command on = new LightOnCommand(light);
		Command off = new LightOffCommand(light);
		Switch sw = new Switch(on, off);

		sw.turnOn();
		sw.turnOff();
	}
}
```

Output:

```
Light is ON
Light is OFF
```

Example2: Remove Control

```
package com.mannemlokesh.example2;

interface Command {
	void execute();
}

class TV {
	public void powerOn() {
		System.out.println("TV is ON");
	}

	public void powerOff() {
		System.out.println("TV is OFF");
	}
}

class TVOnCommand implements Command {
	private TV tv;
```

```
	public TVOnCommand(TV tv) {
		this.tv = tv;
	}

	public void execute() {
		tv.powerOn();
	}
}

class TVOffCommand implements Command {
	private TV tv;

	public TVOffCommand(TV tv) {
		this.tv = tv;
	}

	public void execute() {
		tv.powerOff();
	}
}

class RemoteControl {
	private Command command;

	public void setCommand(Command command) {
		this.command = command;
	}

	public void pressButton() {
		command.execute();
	}
}

public class Example2TVRemoteClient {
	public static void main(String[] args) {
		TV tv = new TV();
		RemoteControl remote = new RemoteControl();

		remote.setCommand(new TVOnCommand(tv));
		remote.pressButton();

		remote.setCommand(new TVOffCommand(tv));
		remote.pressButton();
	}
}
```

Output:

```
TV is ON
TV is OFF
```

Example3: Command Queue with Undo

```
package com.mannemlokesh.example3;

interface Command {
	void execute();
```

```
	void undo();
}

class Document {
	public void write(String content) {
		System.out.println("Writing: " + content);
	}

	public void erase(String content) {
		System.out.println("Erasing: " + content);
	}
}

class WriteCommand implements Command {
	private Document doc;
	private String text;

	public WriteCommand(Document doc, String text) {
		this.doc = doc;
		this.text = text;
	}

	public void execute() {
		doc.write(text);
	}

	public void undo() {
		doc.erase(text);
	}
}

public class Example3CommandQueueClient {
	public static void main(String[] args) {
		Document doc = new Document();
		Command write = new WriteCommand(doc, "Hello World");

		write.execute();
		write.undo();
	}
}
```

Output:

```
Writing: Hello World
Erasing: Hello World
```

17.9 Command vs Other Patterns

Pattern	Purpose
Command	Encapsulate a request as an object
Strategy	Encapsulate a family of algorithms
Observer	Notify dependent objects of changes
Chain	Pass request through a chain of handlers

17.10 Pros and Cons

✅ **Pros**

- Decouples invoker and receiver
- Supports undo/redo/logging
- Easy to add new commands

❌ **Cons**

- Increases number of classes
- Can be overkill for simple operations

17.11 When to Use / Not to Use

Use When:

- Need undo/redo functionality
- You want to queue requests
- You want to decouple sender and receiver

Avoid When:

- Logic is very simple
- Overhead of extra classes isn't justified

17.12 Summary

Feature	Description
Intent	Encapsulate request as object
Use Case	UI buttons, undo/redo, queuing
Key Benefit	Command decouples object and operations

The **Command Pattern** is powerful for queuing, logging, undo/redo, and cleanly separating invocation from execution.